

Air University Aerospace And Aviation Campus Kamra

Department of Computer Science



Sequence Diagrams

Name: Haleema Abid, Samreen Shoukat, Hamna Saleem

ID: 245053, 245077, 245047

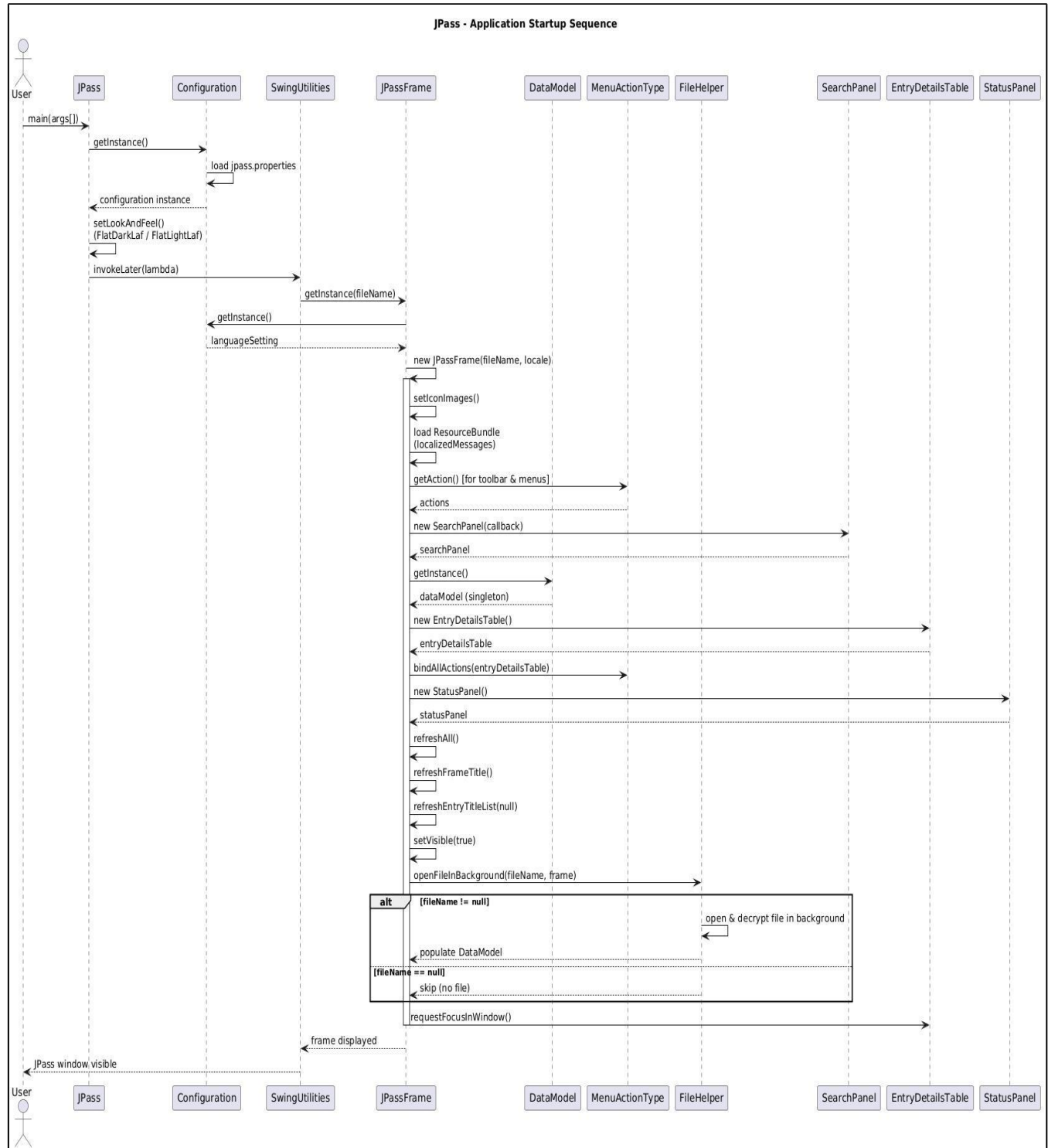
Semester: BSCYS-4B

Submitted To: Sir Zakaullah

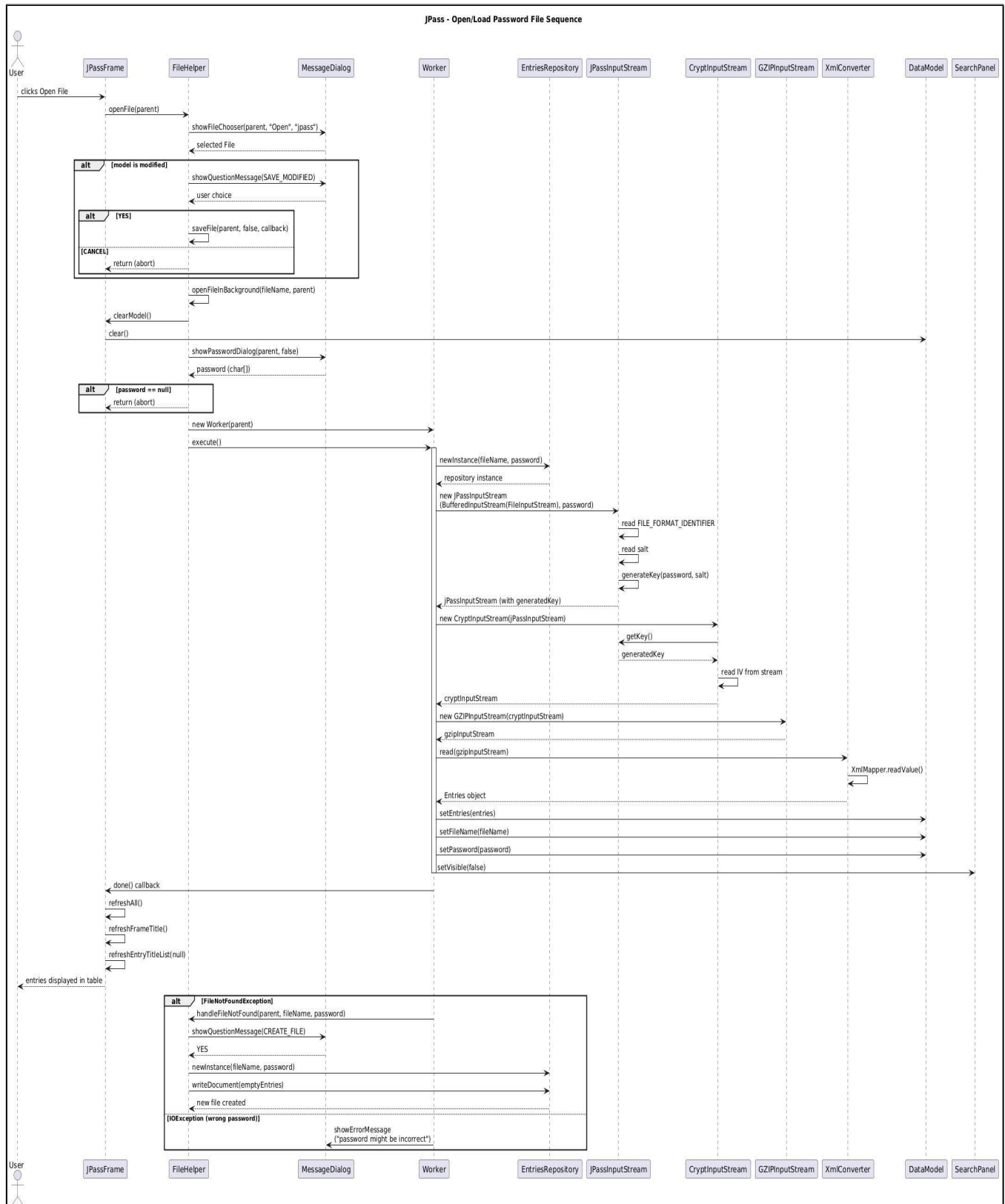
Date: March 16, 2026

Sequence Diagrams:

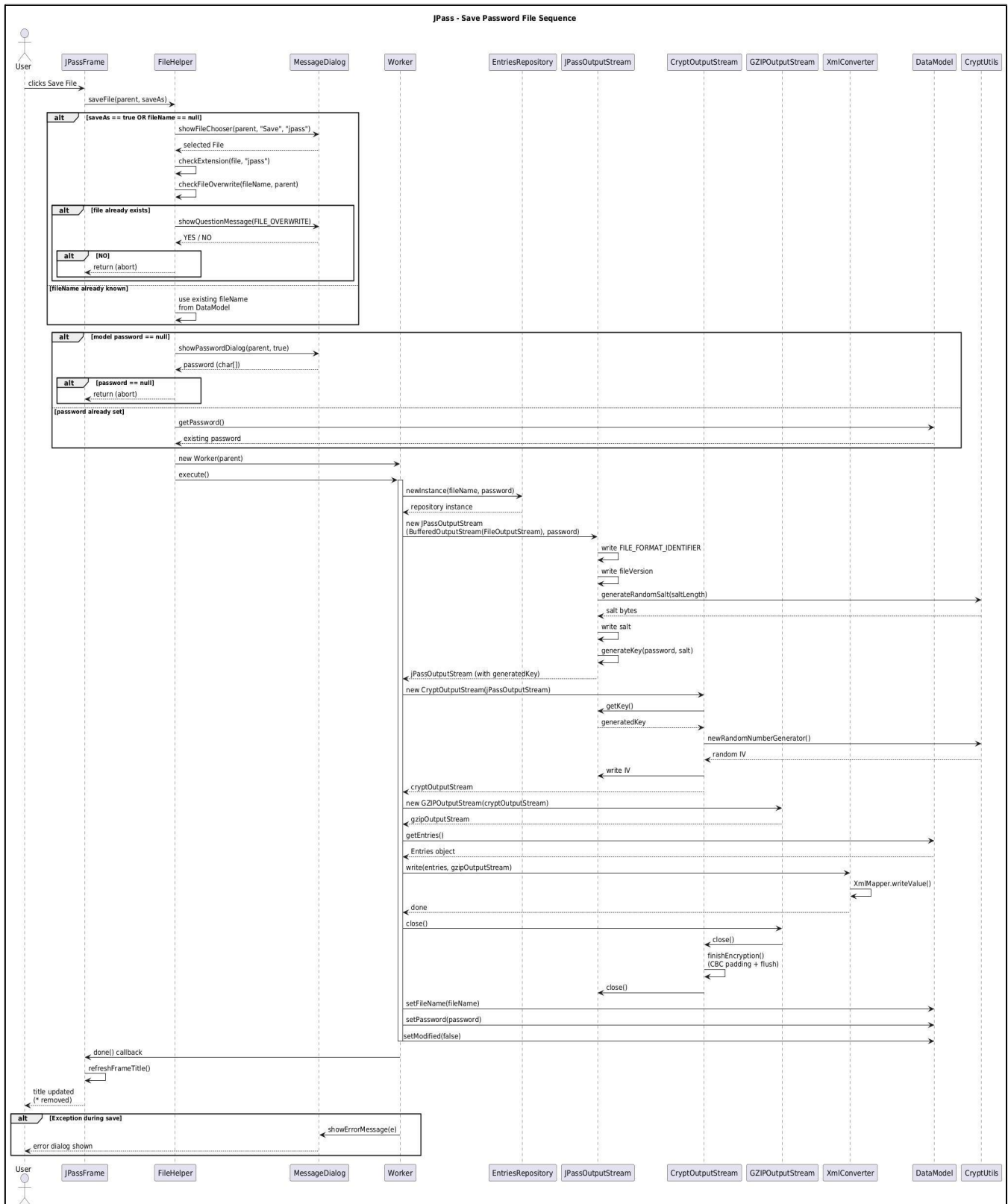
- Application Startup



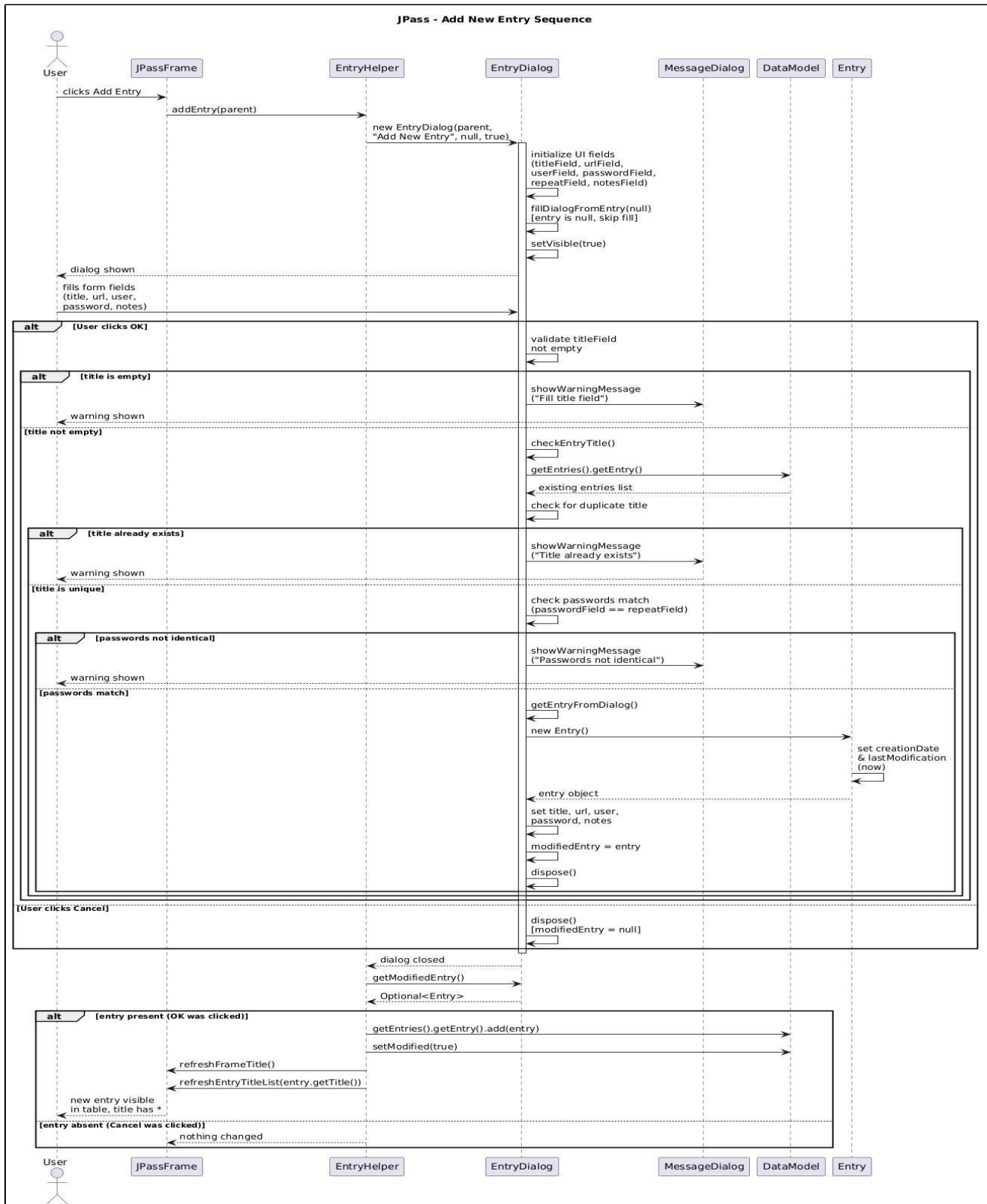
• Open Password File



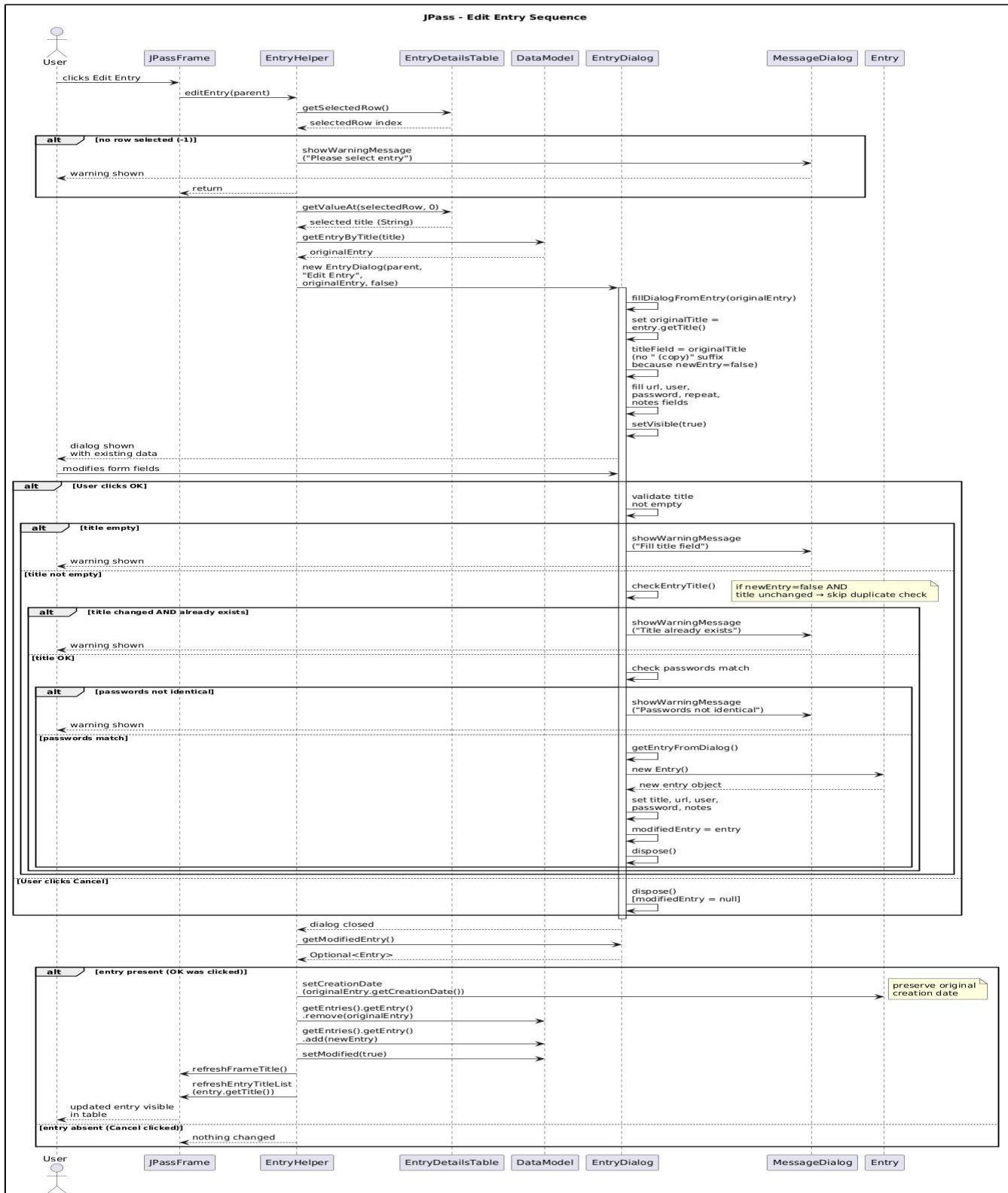
- Save Password File



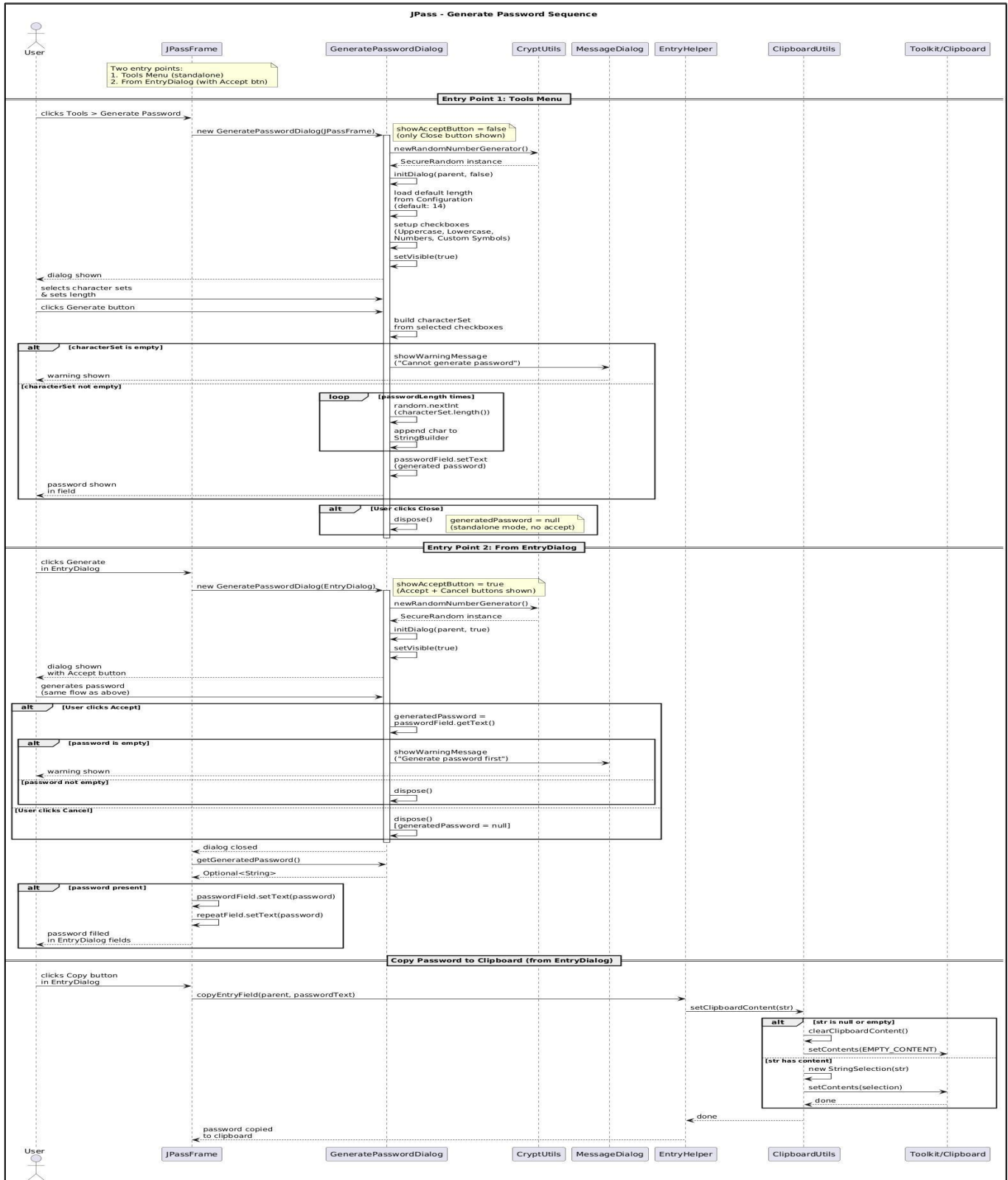
- Add Entry



• Edit Entry



• Generate Password



Analysis:

JPass is a local desktop password manager that stores encrypted credentials in .jpass files. Since it has no network login or user accounts, its entire security model is built around **file-based authentication** and **data-level authorization** — meaning only the user who knows the master password can decrypt and access the stored entries. This analysis examines how JPass implements these mechanisms through its source code.

1. Authentication Mechanisms

1.1 Master Password as Authentication Factor

The primary authentication mechanism in JPass is a single master password that the user must provide every time they open or save a .jpass file. This is handled in FileHelper.java:

- Password is collected as `char[]` — not `String` — so it can be explicitly zeroed from memory after use, unlike `String` objects which are immutable and linger in the JVM string pool
- If the user cancels or provides no password, the operation is immediately aborted
- The password is never logged, printed, or stored in plaintext at any point in the application
- This single master password acts as the sole gatekeeper to all stored entries

1.2 Key Derivation — PBKDF2-HMAC-SHA256

JPass does not use the raw password directly for encryption. Instead, it derives a strong cryptographic key using PBKDF2 with HMAC-SHA256, implemented in CryptUtils.java:

- **Algorithm:** PBKDF2-HMAC-SHA256 — an industry-standard password-based key derivation function
- **Iterations:** 310,000 — directly aligned with OWASP 2021 recommendations for this algorithm
- **Output Key Length:** 256 bits — perfectly sized for AES-256 encryption
- **Purpose:** Transforms the human-memorable password into a cryptographically strong key while making brute-force attacks computationally very expensive

- Even if an attacker obtains the .jpass file, they must perform 310,000 hash operations per password guess, making large-scale attacks impractical

1.3 Random Salt Generation

A unique cryptographic salt is generated for every file save operation, implemented in JPassOutputStream.java and CryptUtils.java:

- Uses SecureRandom — a cryptographically strong random number generator backed by OS-level entropy
- Falls back to java.util.Random only if SecureRandom is completely unavailable (an exceptional edge case)
- The salt is written into the file header in plaintext — this is by design, salts are not secret
- Even if two files are protected by the same password, their different salts produce completely different derived keys and ciphertexts
- This defeats rainbow table attacks and pre-computation attacks entirely

1.4 Implicit Authentication via Decryption Verification

JPass does not store a password hash or verification token. Authentication success or failure is determined implicitly through the decryption process itself:

- Correct password → correct key derived → decryption succeeds → entries are loaded
- Wrong password → wrong key derived → CBC padding verification fails → DecryptException is thrown → IOException("can't decrypt") is raised

This is handled in FileHelper.java

2. Encryption Architecture

2.1 AES-256 in CBC Mode

All entry data is encrypted using AES-256 in Cipher Block Chaining (CBC) mode, implemented across Aes256.java and Cbc.java. The full encryption pipeline in EntriesRepository.java is:

The layered stack from top to bottom:

- XmlConverter — serializes Java Entry objects into XML format
- GZIPOutputStream — compresses the XML data before encryption
- CryptOutputStream → Cbc → Aes256 — encrypts the compressed data using AES-256-CBC
- JPassOutputStream — writes the file header (identifier, version, salt)
- FileOutputStream — writes raw bytes to disk

2.2 Initialization Vector (IV) Handling

A fresh random IV is generated for every encryption operation in CryptOutputStream.java:

- A 16-byte (128-bit) IV is generated using SecureRandom
- The IV is written to the output stream before the encrypted data so it can be recovered during decryption
- A new IV each time ensures that encrypting identical data twice produces completely different ciphertexts, preventing pattern analysis

2.3 CBC Padding and Integrity Check

Cbc.java applies PKCS-style padding to the final block during encryption:

- Invalid padding (caused by wrong key or file corruption) throws DecryptException
- This serves as a basic integrity check, tampered or corrupted files will fail to decrypt
- This is the mechanism that makes implicit authentication possible

3. Authorization Mechanisms

3.1 File-Level Authorization Model

JPass implements implicit file-level authorization. There is no separate authorization layer — authentication and authorization are unified:

- A user who successfully decrypts the file is automatically authorized to read, add, edit, and delete all entries within it
- There are no roles, permissions, or access levels — it is an all-or-nothing model
- This is appropriate and intentional for a single-user local desktop application where the threat model is unauthorized file access, not multi-user privilege separation

3.2 In-Memory Password Retention for Session

Once the user authenticates, the password is retained in `DataModel.java` for the duration of the session:

```
private char[] password = null;
```

- Stored as `char[]` — allows the application to zero it out when no longer needed
- Reused for subsequent save operations without re-prompting the user every time
- Cleared automatically when `DataModel.clear()` is called (opening a new file or closing)
- This is a deliberate usability vs. security trade-off, keeping the password in memory avoids repeated prompting while the file is being worked on

3.3 Modified State Tracking and Data Protection

JPass tracks unsaved changes to prevent accidental data loss, in `JPassFrame.java`:

- An asterisk `*` is prepended to the window title whenever unsaved changes exist
- User is always prompted before closing with unsaved data
- This indirectly enforces data integrity — modified but unencrypted in-memory state is never silently discarded.